

Rapport Mini-Projet : Wordle Solver

Fissah Khadidja
Matricule: 232331440603

Table des matières

0.1	Introduction	2
0.2	Analyse du Problème	2
0.2.1	Algorithme et Structure	2
0.2.2	Justification des Structures de Données	2
0.2.3	Analyse de Complexité	2
0.3	Code Source	2
0.4	Conclusion	6

0.1 Introduction

Ce projet consiste à développer un programme en C structuré séparant la logique du jeu, l'intelligence artificielle et l'affichage. L'objectif est de permettre à un utilisateur de jouer ou de laisser un solveur automatique trouver le mot secret en optimisant ses essais.

0.2 Analyse du Problème

Le programme utilise une structure modulaire pour résoudre le jeu Wordle.

0.2.1 Algorithme et Structure

L'application repose sur trois piliers :

- **Moteur** : Compare les mots et renvoie les états (CORRECT, MAL_PLACE, ABSENT).
- **Solveur** : Utilise une **liste chaînée dynamique** pour stocker les mots possibles et les filtrer par récursivité.
- **Interface** : Gère les entrées utilisateur et l'affichage des résultats.

0.2.2 Justification des Structures de Données

Nous avons utilisé deux structures complémentaires :

- **Un Tableau** : Utilisé pour stocker de manière permanente le dictionnaire chargé depuis le fichier. Il permet un accès aléatoire immédiat en $O(1)$ pour choisir le mot secret.
- **Une Liste Chaînée** : Utilisée par le solveur comme structure de travail. Elle permet de supprimer efficacement les mots impossibles en libérant la mémoire (**free**) sans avoir à décaler les éléments restants, contrairement à un tableau.

0.2.3 Analyse de Complexité

- **Spatiale** : La complexité est $O(N)$ où N est le nombre de mots dans le dictionnaire, car chaque mot occupe un nœud de la liste.
- **Temporelle** : Le filtrage du dictionnaire parcourt la liste une fois par tour, soit une complexité de $O(N)$. Sur un dictionnaire réduit, le solveur est quasi-instantané.

0.3 Code Source

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5 #include <time.h>
6
7 #define TAILLE_MOT 5
8
9 // --- STRUCTURES ---
10 typedef struct Element {
```

```

11     char mot[6];
12     struct Element* suivant;
13 } Element;
14
15 typedef enum { ABSENT, MAL_PLACE, CORRECT } EtatLettre;
16
17 typedef struct {
18     EtatLettre resultats[TAILLE_MOT];
19 } Analyse;
20
21 // --- FONCTIONS DE GESTION DU DICTIONNAIRE ---
22 Element* ajouter_au_debut(Element* liste, const char* m) {
23     Element* nouveau = malloc(sizeof(Element));
24     if(!nouveau) return liste;
25     strncpy(nouveau->mot, m, 5);
26     nouveau->mot[5] = '\0';
27     nouveau->suivant = liste;
28     return nouveau;
29 }
30
31 // NOUVEAU : Charge les mots depuis le fichier word.txt
32 Element* charger_depuis_fichier(const char* nom_fichier, int*
    nb_mots) {
33     FILE* f = fopen(nom_fichier, "r");
34     if (!f) {
35         printf("Erreur : Impossible de trouver le fichier %s\n",
            nom_fichier);
36         return NULL;
37     }
38     Element* liste = NULL;
39     char tampon[10];
40     *nb_mots = 0;
41     while (fscanf(f, "%s", tampon) == 1) {
42         if (strlen(tampon) == 5) {
43             liste = ajouter_au_debut(liste, tampon);
44             (*nb_mots)++;
45         }
46     }
47     fclose(f);
48     return liste;
49 }
50 void copier_liste_vers_tableau(Element* L, char tab[][6]) {
51     int i = 0;
52     while (L) {
53         strcpy(tab[i++], L->mot);
54         L = L->suivant;
55     }
56 }
57
58 // --- LOGIQUE DU JEU ---
59

```

```

60 Analyse comparer_mots(const char* essai, const char* cible) {
61     Analyse bilan;
62     bool cible_utilisee[5] = {false}, essai_utilise[5] = {false};
63     for (int i = 0; i < 5; i++) {
64         if (essai[i] == cible[i]) {
65             bilan.resultats[i] = CORRECT;
66             cible_utilisee[i] = essai_utilise[i] = true;
67         } else {
68             bilan.resultats[i] = ABSENT;
69         }
70     }
71     for (int i = 0; i < 5; i++) {
72         if (essai_utilise[i]) continue;
73         for (int j = 0; j < 5; j++) {
74             if (!cible_utilisee[j] && essai[i] == cible[j]) {
75                 bilan.resultats[i] = MAL_PLACE;
76                 cible_utilisee[j] = true;
77                 break;
78             }
79         }
80     }
81     return bilan;
82 }
83
84 bool verifier_compatibilite(const char* mot_candidat, const char*
dernier_essai, Analyse bilan_obtenu) {
85     Analyse test = comparer_mots(dernier_essai, mot_candidat);
86     for (int i = 0; i < 5; i++) {
87         if (test.resultats[i] != bilan_obtenu.resultats[i]) return
false;
88     }
89     return true;
90 }
91
92 Element* filtrer_dictionnaire(Element* L, const char* essai,
Analyse bilan) {
93     if (L == NULL) return NULL;
94     L->suivant = filtrer_dictionnaire(L->suivant, essai, bilan);
95     if (!verifier_compatibilite(L->mot, essai, bilan)) {
96         Element* reste = L->suivant;
97         free(L);
98         return reste;
99     }
100     return L;
101 }
102
103 void afficher_bilan_textuel(const char* essai, Analyse bilan) {
104     const char* texte_etat[] = {"ABSENTE", "MAL PLACEE", "CORRECTE"};
105     printf("\nAnalyse de '%s' :\n", essai);
106     for(int i = 0; i < 5; i++) {

```

```

107         printf("    - %c : %s\n", essai[i], texte_etat[bilan.
           resultats[i]]);
108     }
109 }
110
111 void liberer_memoire(Element* L) {
112     while(L) {
113         Element* a_supprimer = L;
114         L = L->suivant;
115         free(a_supprimer);
116     }
117 }
118
119 // --- MAIN (Conserve ta structure d'origine) ---
120 int main() {
121     srand(time(NULL));
122     char rejouer;
123
124     // Initialisation du dictionnaire depuis le fichier
125     int nombre_total_mots = 0;
126     Element* dictionnaire_complet = charger_depuis_fichier("word.
           txt", &nombre_total_mots);
127
128     if (!dictionnaire_complet) return 1;
129     char mots_pour_tirage[nombre_total_mots][6];
130     copier_liste_vers_tableau(dictionnaire_complet,
           mots_pour_tirage);
131
132     do {
133         char* mot_secret = mots_pour_tirage[rand() %
           nombre_total_mots];
134         int choix_mode;
135
136         printf("\n===== WORDLE=====");
137         printf("\n1. Mode Joueur (Humain)");
138         printf("\n2. Mode Solveur (Automatique)");
139         printf("\nVotre choix : ");
140         scanf("%d", &choix_mode);
141
142         if (choix_mode == 1) {
143             char tentative[10];
144             bool gagne = false;
145             for (int tour = 1; tour <= 6; tour++) {
146                 printf("\nTour %d/6. Entrez un mot (MAJUSCULES) : "
           , tour);
147                 scanf("%s", tentative);
148                 Analyse res = comparer_mots(tentative, mot_secret);
149                 afficher_bilan_textuel(tentative, res);
150                 if (strcmp(tentative, mot_secret) == 0) {
151                     printf("\n>>> BRAVO ! Vous avez gagne.\n");
152                     gagne = true; break;

```

```

153     }
154 }
155 if(!gagne) printf("\n>>> PERDU ! Le mot etait : %s\n",
    mot_secret);
156 }
157 else {
158     Element* possibles = NULL;
159     for (int i = 0; i < nombre_total_mots; i++)
160         possibles = ajouter_au_debut(possibles,
            mots_pour_tirage[i]);
161
162     for (int tour = 1; tour <= 6; tour++) {
163         if (!possibles) break;
164         char* essai_ia = possibles->mot;
165         Analyse res = comparer_mots(essai_ia, mot_secret);
166         printf("\nTour %d. L'IA tente : %s", tour, essai_ia
            );
167         afficher_bilan_textuel(essai_ia, res);
168         if (strcmp(essai_ia, mot_secret) == 0) {
169             printf("\n>>> L'IA a gagne en %d tours !\n",
                tour);
170             break;
171         }
172         possibles = filtrer_dictionnaire(possibles,
            essai_ia, res);
173     }
174     liberer_memoire(possibles);
175 }
176
177 printf("\nVoulez-vous refaire une partie ? (o/n) : ");
178 scanf(" %c", &rejouer);
179
180 } while (rejouer == 'o' || rejouer == 'O');
181
182 liberer_memoire(dictionnaire_complet);
183 printf("\nMerci et au revoir !\n");
184 return 0;
185 }

```

0.4 Conclusion

efficacité des structures de données dynamiques pour résoudre des problèmes de logique complexes. L'utilisation du dictionnaire externe rend le programme flexible et évolutif.